

EXPERIMENT – 3

➤ Fundamentals of Errors and Debugging:

- **Error:**

An **error** is simply a mistake made by the programmer while writing the code. It can happen because of a typing mistake, wrong syntax, or using the wrong formula or logic.

- **Bug:**

A **bug** is a problem in a program that makes it behave differently from what we expect. In simple words, the program may run, but it gives the **wrong result or does something unexpected**.

- **Debugging:**

1. **Debugging** is the process of **finding and fixing bugs in a program**. Instead of randomly changing the code, the programmer carefully checks what went wrong, finds the cause, fixes it, and then tests the program again.

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	<code>int a = 10 (missing ;)</code>
Compilation / Type Errors	Operations are performed on incompatible types or required declarations are missing.	Assigning a string to an integer/pointer incorrectly, using an undeclared variable.	<code>int *p = "hello";</code>
Linker Errors	The compiler generates object files, but the linker cannot combine them into a complete executable program.	Function declared but not defined, or missing required libraries.	undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally while it is running.	Division by zero, accessing invalid memory addresses, etc.	<code>int x = 10 / 0;</code>
Logical Errors	The program runs successfully, but produces an incorrect result.	Wrong formula, incorrect condition, or incorrect loop logic.	<code>for(i=0; i<5; i++) instead of</code>

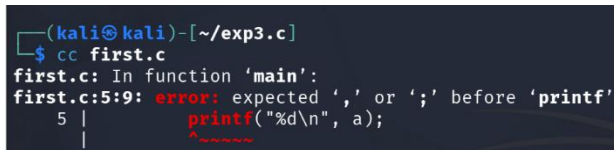
Why Does a Program Compile Successfully but Produce Incorrect Output?

A program can compile successfully but still produce incorrect output because the compiler mainly checks syntax and type-related errors, not whether the programmer's logic is correct. For example, using `+` instead of `*` may be valid C code, but it gives the wrong result. This is called a **logical error**.

Case 1: Syntax & Compilation Errors:

Example code-

```
//first.c with bug
#include<stdio.h>
int main()
{
    int a = 4
    printf("%d\n", a);
    return 0;
}
```



```
(kali㉿kali)-[~/exp3.c]
$ cc first.c
first.c: In function 'main':
first.c:5:9: error: expected ',', or ';' before 'printf'
   5 |         printf("%d\n", a);
     |         ~~~~~
```

Debugging-

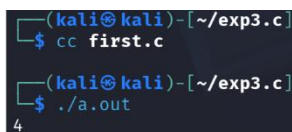
The compiler reaches the printf statement and gets confused because the previous statement is not finished. In C, every variable declaration or statement usually ends with a semicolon (;).

So the error is:

Syntax Error → Missing semicolon after int a = 4.

Correct code-

```
//first.c with correction
#include<stdio.h>
int main()
{
    int a = 4;
    printf("%d\n", a);
    return 0;
}
```



```
(kali㉿kali)-[~/exp3.c]
$ cc first.c
(kali㉿kali)-[~/exp3.c]
$ ./a.out
4
```

Case 2: Compiler Warnings (-Wall, -Wextra):

Example code-

```
#include<stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

```
(kali㉿kali)-[~/exp3.c]
$ cc second.c

(kali㉿kali)-[~/exp3.c]
$ cc -Wall -Wextra -g second.c
second.c: In function 'main':
second.c:4:13: warning: unused variable 'a' [-Wunused-variable]
    4 |         int a;
      |         ^
```

Debugging-

Here `int a;` is valid, but the variable `a` is never used. A compiler may show a warning about this. So, there is no syntax, compilation, linker, runtime, or logical error here. The only possible warning is that `a` is declared but not used.

Correct code-

```
#include<stdio.h>
int main(void)
{
    printf("Hello\n");
    return 0;
}
```

Case-3: printf-Based Debugging (Logical bug):

Example code-

```
#include<stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("Final Sum = %d\n", sum);
    return 0;
}
```

```
(kali㉿kali)-[~/exp3.c]
$ cc third.c

(kali㉿kali)-[~/exp3.c]
$ ./a.out
DEBUG: enetering iteration i = 0
DEBUG: sum update to = 0
DEBUG: enetering iteration i = 0
DEBUG: sum update to = 1
DEBUG: enetering iteration i = 1
DEBUG: sum update to = 3
DEBUG: enetering iteration i = 3
DEBUG: sum update to = 6
DEBUG: enetering iteration i = 6
DEBUG: sum update to = 10
Final Sum = 10
```

Debugging-

The above program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug. Debug `printf()` statements help expose the exact iteration behaviour.

Correct code-

```
#include<stdio.h>

int main(void)
{
int i, sum = 0;
for(i = 0; i <= 5; i++)
{
printf("DEBUG: entering iteration i = %d\n", i);
sum = sum + i;
printf("DEBUG: sum updated to = %d\n", sum);
}
printf("Final Sum = %d\n", sum);
return 0;
}
```



```
(kali@kali)~[/exp3.c]
$ ./a.out
DEBUG: entering iteration i = 0
DEBUG: sum update to = 0
DEBUG: entering iteration i = 0
DEBUG: sum update to = 1
DEBUG: entering iteration i = 1
DEBUG: sum update to = 3
DEBUG: entering iteration i = 3
DEBUG: sum update to = 6
DEBUG: entering iteration i = 6
DEBUG: sum update to = 10
DEBUG: entering iteration i = 10
DEBUG: sum update to = 15
Final Sum = 15
```

➤ Introduction to GDB (GNU Debugger):

What is GNU and GDB?

GNU: GNU stands for “**GNU’s Not Unix!**” It is a free and open-source software project. Its aim was to create a completely free Unix-like operating system and provide users with software they could freely use, modify, and share.

GDB (GNU Debugger): GDB stands for **GNU Debugger**. It is a powerful debugging tool used by programmers to find and fix errors in programs. With GDB, we can **pause a program using breakpoints, check variable values, execute code step by step, and investigate program crashes**.

In simple terms, **GNU provides many important development tools, while GDB helps programmers understand and fix problems in their programs.**

Installation & Compilation for GDB-

```
# Update package list and install GDB on Linux systems
$ sudo apt update
$ sudo apt install gdb
```

```
# Compile source code with debugging symbols enabled (-g)
$ cc -g -Wall -Wextra program.c -o program
```

```
# Launch GDB with the compiled binary
$ gdb ./program
```

Case 4: Debugging a Logical Error using GDB:

Example code-

```
#include<stdio.h>           // Line 1
int main()                 // Line 2
{                           // Line 3
    int i, sum = 0;         // Line 4
    for(i = 1; i < 5; i++)   // Line 5 (Logical Bug: should be i <= 5)
    {                       // Line 6
        sum = sum + i;       // Line 7
    }                       // Line 8
    printf("Sum = %d\n", sum); // Line 9
    return 0;               // Line 10
}                           // Line 11
```

```
(kali@kali)~/exp3.c
$ cc fourth.c

(kali@kali)~/exp3.c
$ ./a.out
Sum = 10
```

Debugging Walkthrough in GDB:

```
(kali@kali)~/exp3.c
$ cc -g fourth.c -o fourth

(kali@kali)~/exp3.c
$ gdb ./fourth
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./fourth...
(gdb) break 7
✗ Undefined command: "break?". Try "help".
(gdb) break 7
Breakpoint 1 at 0x1151: file fourth.c, line 7.
(gdb) run
Starting program: /home/kali/exp3.c/fourth
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at fourth.c:7
7          sum = sum + i;           //line7....
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at fourth.c:7
7          sum = sum + i;           //line7
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at fourth.c:7
7          sum = sum + i;           //line7
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at fourth.c:7
7          sum = sum + i;           //line7
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
[Inferior 1 (process 8031) exited normally]
(gdb) continue
```

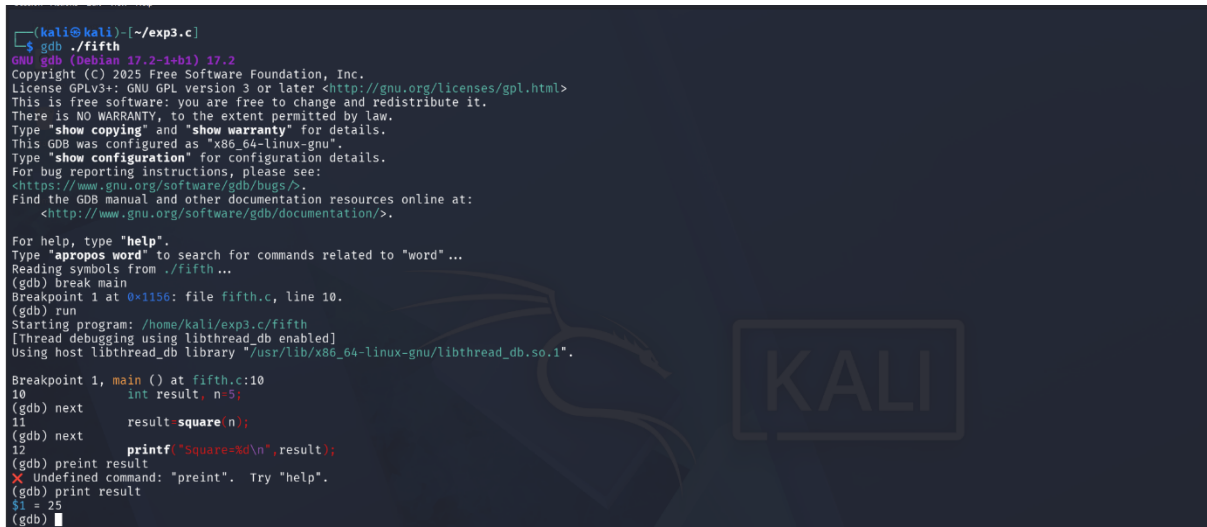
Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from `i < 5` to `i <= 5`.

Case 5: Debugging Functions (next vs. step):

Example code-

```
#include<stdio.h>
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Option A: Stepping Over with next-



```
(kali@kali)~/exp3.c
$ gdb ./fifth
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./fifth ...
(gdb) break main
Breakpoint 1 at 0x1156: file fifth.c, line 10.
(gdb) run
Starting program: /home/kali/exp3.c/fifth
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at fifth.c:10
10      int result, n = 5;
(gdb) next
11      result=square(n);
(gdb) next
12      printf("Square=%d\n",result);
(gdb) print result
Undefined command: "preint". Try "help".
(gdb) print result
$1 = 25
(gdb)
```

Option B: Stepping Into with step-

```
(kali@kali)-[~/exp3.c]
$ gdb ./fifth
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./fifth...
(gdb) break main
Breakpoint 1 at 0x1156: file fifth.c, line 10.
(gdb) run
Starting program: /home/kali/exp3.c/fifth
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at fifth.c:10
10      int result, n=5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at fifth.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result1
$2 = 25
(gdb)
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt):

Example code-

```
#include<stdio.h>
int divide(int a, int b)
{
    return a / b;           // Division by zero when b = 0
}
int calculate(int x)
{
    return divide(x, 0);     // Passes 0 as divisor
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

```

(kali@kali)-[~/exp3.c]
$ cc -g sixth.c -o sixth
(kali@kali)-[~/exp3.c]
$ gdb ./sixth
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sixth...
(gdb) run
Starting program: /home/kali/exp3.c/sixth
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at sixth.c:4
4      return a/b;                                //Division by zero when b=0
(gdb) backtrace
#0 0x000055555555147 in divide (a=10, b=0) at sixth.c:4
#1 0x000055555555166 in calculate (x=10) at sixth.c:8
#2 0x00005555555517a in main () at sixth.c:12
(gdb) frame 0
#0 0x000055555555147 in divide (a=10, b=0) at sixth.c:4
4      return a/b;                                //Division by zero when b=0
(gdb) print b
$b = 0
(gdb) frame 1
#1 0x000055555555166 in calculate (x=10) at sixth.c:8
8      return divide(x,0);                        //Passes 0 as divisor
(gdb) print x
$x = 10

```

Understanding Stack Frames-

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

Case 7: Debugging Array Out-of-Bounds Errors using GDB:

Example code-

```

#include<stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]); //Line 8
    }

    return 0;
}

```


Inspection in GDB-

```
(kali@kali)-[~]
└─$ cc -g seventh.c -o seventh

(kali@kali)-[~]
└─$ gdb ./seventh
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./seventh...
(gdb) break 8
✗ Undefined command: "break8". Try "help".
(gdb) break 8
Breakpoint 1 at 0x114a: file seventh.c, line 8.
(gdb) run
Starting program: /home/kali/seventh
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at seventh.c:8
8      scanf("%d",&a[i]);           //Line8
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd30
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd40
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd44
(gdb)
```

Correct Loop Condition: for(i = 0; i < 5; i++)

Case 8: Segmentation Faults (SIGSEGV) using GDB:

Example code-

```
#include<stdio.h>
int main()
{
    int *p = NULL;           // Pointer points to address 0x0
    *p = 10;                 // Attempting to write to address 0x0 causes
                             // Segmentation Fault
    printf("%d\n", *p);
    return 0;
}
```

Debugging in GDB-

```
(kali@kali)~$ cc -g sample8.c -o sample8
(kali@kali)~$ gdb ./sample8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample8...
(gdb) run
Starting program: /home/kali/sample8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample8.c:5
5      *p = 10;           //attempting to write to address 0x0 causes
    segmentation fault
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
X Cannot access memory at address 0x0
(gdb)
```

Corrected code-

```
#include<stdio.h>
int main()
{
    int x;
    int *p = &x;           // Assign p to point to valid memory address of x
    *p = 10;               // Valid write
    printf("%d\n", *p);
    return 0;
}
```

```
(kali@kali)~$ cc sample8.c
(kali@kali)~$ ./a.out
10
```

➤ Systematic Debugging Strategy:

A **systematic debugging strategy** is a structured approach used to identify, understand, and fix errors in a program instead of making random changes. It helps programmers solve problems efficiently and avoid introducing new errors.

Steps in Systematic Debugging

1. Reproduce the Error

Run the program and identify exactly when and how the error occurs. Note the input, output, and error messages.

2. Understand the Problem

Determine what the program is **expected to do** and compare it with what it is **actually doing**.

Use debugging tools such as **GDB**, breakpoints, watchpoints, and variable inspection to find the part of the program causing the problem.

Based on the evidence, identify the most likely cause of the error. Avoid changing multiple things at once.

Use debugging commands, additional output statements, or test cases to confirm whether the suspected cause is correct.

Make the smallest appropriate change to correct the problem.

Run the program with the original input and additional test cases to verify that the error has been fixed.

Make sure the fix has not caused new errors or affected other parts of the program.

➤ Error Classifications-

A syntax error occurs when the code violates the rules of the C language.

Error: Missing ; after the printf() statement.

A compilation (type) error occurs when incompatible data types are used.

Error: Cannot assign a string (char *) to an int.

A linker error occurs when a function is declared but its definition is missing.

```
#include <stdio.h>
void display();           // Function declaration
int main()
{
    display();            // Function called
    return 0;
}
```

Error: display() is declared but never defined, resulting in an **undefined reference** during linking.

4. Runtime Error

A runtime error occurs while the program is executing.

```
#include <stdio.h>
int main()
{
    int a = 10, b = 0;
    int result = a / b;           // Division by zero
    printf("%d", result);
    return 0;
}
```

Error: Division by zero causes the program to crash or produce undefined behavior at runtime.

5. Logical Error

A logical error occurs when the program runs successfully but produces incorrect output.

```
#include <stdio.h>
int main()
{
    int length = 10, width = 5;
    int area = 2 * (length + width);           // Incorrect formula
    printf("Area = %d", area);
    return 0;
}
```

Output:

Area = 30

Correct Logic:

area = length * width;

Correct Output:

Area = 50

➤ GNU Concept-

GNU stands for “**GNU’s Not Unix.**” Its main goal is to provide a complete Unix-like operating system and a collection of software that users can freely use, study, modify, and share.

The GNU ecosystem contains many important tools used in software development, such as:

- **GCC (GNU Compiler Collection):** Compiles programs written in languages such as C and C++.
- **GDB (GNU Debugger):** Helps programmers find and fix errors in programs.
- **GNU Make:** Automates the process of compiling and building software.
- **GNU Binutils:** Provides tools for assembling, linking, and working with binary files.
- **GNU Core Utilities:** Provides common command-line programs such as ls, cp, mv, and cat.

How GDB Fits into the GNU Ecosystem?

Ans-

GDB (GNU Debugger) is a debugging tool developed as part of the GNU Project. It works closely with compilers such as **GCC**. After compiling a C program with debugging information, GDB allows the programmer to examine and control the program while it is running.

➤ Shell Context-

1. Shell

A **Shell** is a command-line interface that allows the user to interact with the operating system. It accepts commands entered by the user, interprets them, and asks the operating system to execute them.

Example:

```
ls
cd Documents
mkdir test
```

2. Kernel

The **Kernel** is the core part of an operating system. It acts as a bridge between **hardware and software**. It manages important resources such as the CPU, memory, files, and input/output devices.

Shell Description

Bash Bourne Again Shell; the most widely used and commonly found default shell on Linux systems.

Zsh Z Shell; an advanced shell with powerful customization and interactive features.

Fish Friendly Interactive Shell; designed to be easy to use, with features such as autosuggestions and syntax highlighting.

➤ The differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Type	What it does	When it triggers	Common GDB command	Typical use
Breakpoint	Pauses program execution at a specific location	When execution reaches a specified line, function, or address	break	Inspect program flow and variables
Watchpoint	Monitors a variable/expression	When the value of an expression changes; GDB also supports stopping on reads or accesses with rwatch/awatch	watch	Find where a variable is unexpectedly modified
Catchpoint	Monitors specific program events	When a specified event occurs, such as a C++ exception being thrown or a library being loaded	catch	Find event-based problems
Tracepoint	Collects information without normally stopping the program for interactive inspection	When execution reaches a specified location; it briefly collects selected data and lets the program continue	trace	Analyze program behavior, especially in remote/embedded debugging